CS3104 MINI PROJECT FINAL REPORT

Vehicle Parking Management System

Final Project Report

Prepared by

Dinesh Reddy Neela — Roll No. 251138

Ramcharan Tej Vankudoth — Roll No. 251160

Contents

17.1 Introduction and Problem Statement **3**

17.2 Technology Stack **3**

17.3 Scope and Feature Set **3**
17.3.1 Users and Operating Environment 4
17.3.2 Representative API Surface 4

17.4 Object-Oriented Architecture **4**
17.4.1 Object-Oriented Concepts Demonstrated 5
17.4.2 Class Design and Associations 5
17.4.3 Architecture 6
17.4.4 Reservation Sequence Diagram 7
17.4.5 Activity Diagram: Direct Parking 8
17.4.6 Use Case Diagram 8
17.4.7 OOP Code Examples 9

17.5 Database Design and Persistence **12**
17.5.1 Fee Rules 12

17.6 Project Interface and Execution Outputs **12**
17.6.1 Build and Run 22
17.6.2 Project Source Map 23

17.7 Limitations and Future Work **23**

17.8 Conclusion **23**

17.1 Introduction and Problem Statement

Finding and managing a suitable parking space is difficult when availability, vehicle requirements, reservations, and charges are tracked separately. The Vehicle Parking Management System (VPMS), branded as ParkFlow in its web interface, provides a browser-based workflow for drivers and parking administrators. A driver can maintain vehicles, inspect parking availability, reserve a compatible slot, check in, review a session, and leave the facility. Administrators manage users, areas, slots, rates, and maintenance schedules.

The application is implemented as a static HTML/CSS/JavaScript frontend, a C++20 REST backend using Crow, and a PostgreSQL database accessed through libpqxx. The browser communicates with the backend using JSON over HTTP and bearer-session authentication. The backend checks ownership and eligibility while performing state-changing parking operations in database transactions.

The project addresses three related problems: allocating a slot compatible with the vehicle and requested time, preventing conflicting assignments or reservations, and keeping an auditable record of parking sessions and fees. The current implementation supports manual check-in and exit. Automated reservation expiry, QR processing, online payment processing, and advanced reporting are outside the implemented feature set.

17.2 Technology Stack

The project combines browser technologies, a C++ backend, and a relational database. The following list reflects the tools and libraries used by the current source code.

- **Frontend:** HTML5, CSS3, and vanilla JavaScript, served as a browser-based single-page interface.
- **Backend language:** C++20.
- **HTTP framework:** Crow, used to define the REST-style backend routes and host the HTTP server.
- **API format:** HTTP requests and JSON responses, with bearer tokens for authenticated operations.
- **Database:** PostgreSQL, accessed from C++ through libpqxx.
- **Cryptography:** OpenSSL Crypto, linked by the backend for authentication-related cryptographic operations.
- **Build and dependency configuration:** CMake 3.20 or newer; CMake FetchContent obtains Crow, while libpqxx and OpenSSL are required dependencies.

17.3 Scope and Feature Set

- **Accounts and profiles:** Registration, login, logout, profile updates, password updates, bearer sessions, and user/admin role checks.

- **Parking discovery:** Parking areas and slot availability with vehicle type, EV, accessibility, location, and time compatibility.
- **Vehicle management:** Owner-scoped create, read, update, and delete operations; direct parking, fee summary, and checkout for a vehicle.
- **Reservations and sessions:** Availability search, reservation creation and cancellation, manual check-in, active sessions, manual exit, fee summaries, history, and in-app notifications.
- **Administration:** User role/status controls and management of parking areas, slots, rates, and scheduled maintenance.
- **Persistence and consistency:** Relational constraints, parameterized database queries, transaction-protected workflows, and database row locks for conflicting parking operations.

17.3.1 Users and Operating Environment

The primary actors are a registered driver and an administrator. The user interface is served from `frontend/index.html` and its CSS/JavaScript assets. The backend listens on the configured server port (default 3000) and connects to PostgreSQL using environment-based configuration. The source configuration uses CMake, C++20, Crow, libpqxx, and OpenSSL.

17.3.2 Representative API Surface

Table 1 lists representative implemented routes. The source files in `backend/src/routes/` define the full route handlers.

Method	Route	Purpose
POST	/api/auth/register	Create an account and bearer session.
POST	/api/auth/login	Authenticate and return the user profile.
GET/POST	/api/vehicles	List the caller's vehicles or create a vehicle.
GET	/api/reservations/availability	Search slots compatible with a vehicle and time range.
POST	/api/reservations	Create a reservation after validation and conflict checks.
POST	/api/reservations/{id}/check-in	Start a parking session for an eligible reservation.
POST	/api/parking-sessions/{id}/exit	Complete a session and calculate its fee.
GET	/api/history	Return the authenticated user's parking history.
GET/POST/ PUT/DELETE	/api/admin/...	Administrator operations for users and parking resources.

Table 1: Representative implemented REST endpoints.

17.4 Object-Oriented Architecture

The C++ backend uses classes to organize startup, persistence, and route registration. `ParkingApplication` owns the configuration, Crow app, database object, and route controllers. Each controller receives a reference to the shared `Database`, so HTTP

concerns remain grouped by feature while data access and transactional workflows are encapsulated in the database layer.

17.4.1 Object-Oriented Concepts Demonstrated

Table 2 names each object-oriented concept explicitly and points to its implementation in this project. The descriptions reflect the custom C++ source; database rows and JSON records are not described as C++ classes.

Concept	How it appears in this project
Classes and Objects	`ParkingApplication`, `Database`, and the route controller classes define application types. `main()` creates a `ParkingApplication` object.
Encapsulation	Configuration, application state, and controller database references are stored in private members and accessed through public operations.
Abstraction	`ParkingApplication::run()` coordinates setup and server startup; `Database` hides connection and transaction details behind workflow methods.
Composition (Has-a)	`ParkingApplication` owns its configuration, database, Crow app, and route controllers as member objects.
Association	Each route controller keeps a reference to the shared `Database`; this is a non-owning association.
Dependency Injection	`ParkingApplication` constructs the shared database first, then passes it to each controller constructor.
Constructors	Explicit constructors initialize `Database`, `ParkingApplication`, and route controllers from their dependencies.
RAII / Resource Management	Local libpqxx connection and transaction objects release resources at scope exit; uncommitted work rolls back.
Exception Handling	Startup and route/database operations catch exceptions, report diagnostics, and convert failures into a startup exit or HTTP error response.

Table 2: Object-oriented concepts implemented in the C++ backend.

17.4.2 Class Design and Associations

Figure 1 models the principal custom backend classes. A filled diamond marks member ownership (composition); a directed association marks the controller's reference to the shared database. The current code does not define a custom inheritance hierarchy or virtual interface. It uses the concepts summarized in Table 2, without claiming inheritance or runtime polymorphism that is not present in the source.

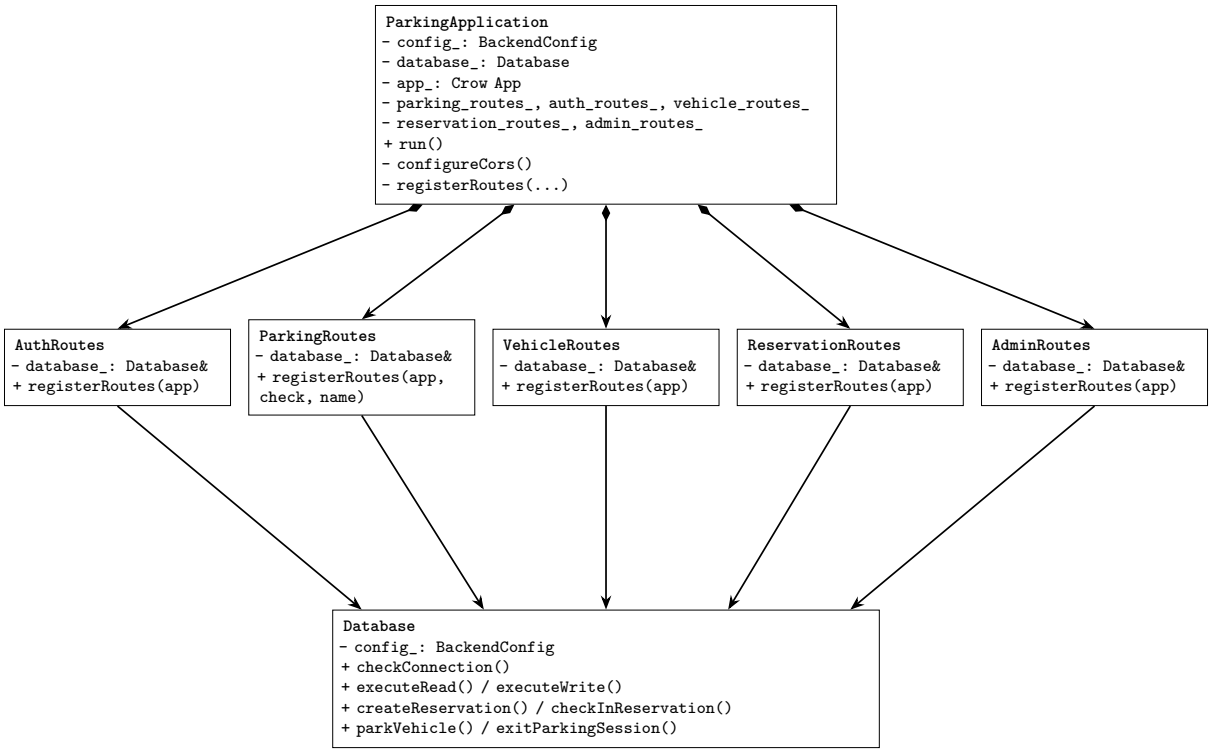


Figure 1: UML class diagram of the custom C++ application structure.

The nine implemented concepts are summarized in Table 2 and demonstrated in Listings 1–5. The project does not use a custom inheritance hierarchy or runtime polymorphism. Records such as users and vehicles are represented by SQL rows and JSON rather than custom C++ domain entity classes.

17.4.3 Architecture

Figure 2 shows the implemented request path. The browser uses the JSON API; Crow route controllers validate and authorize requests; the database layer performs reads and transactional writes against PostgreSQL.

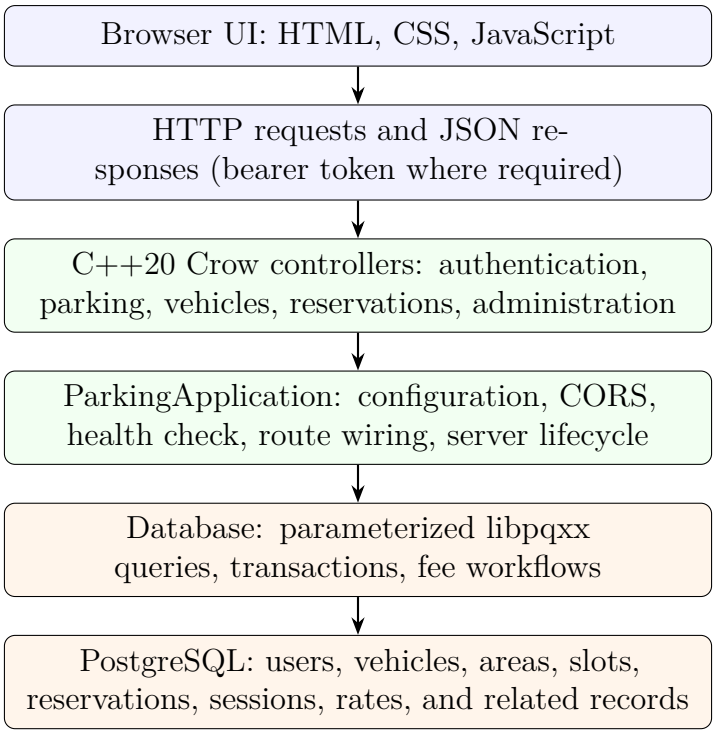


Figure 2: Implemented system architecture and request path.

17.4.4 Reservation Sequence Diagram

Figure 3 shows the principal reservation workflow. The route checks the authenticated identity and request data, then delegates the consistency-sensitive work to the database transaction. The database checks the vehicle, slot, time range, and conflicts before committing the reservation.

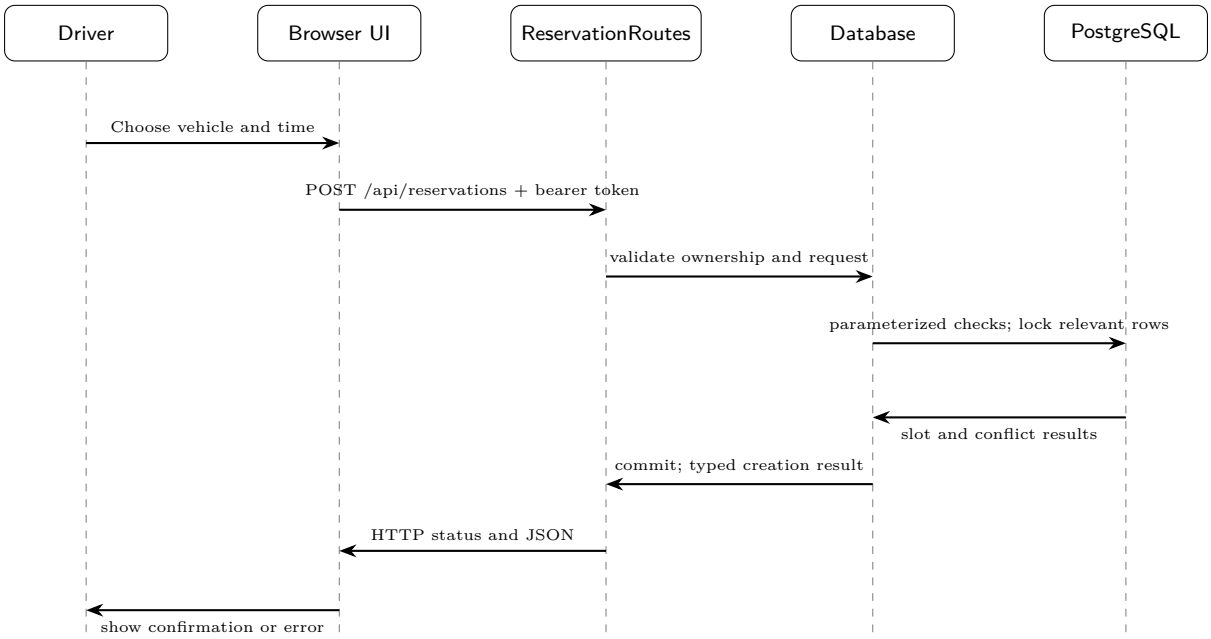


Figure 3: Sequence diagram for reservation creation.

17.4.5 Activity Diagram: Direct Parking

Figure 4 describes direct parking at a compatible slot. The transaction locks the vehicle and slot, checks ownership and eligibility, rejects conflicts, updates slot state, creates the assignment, and commits.

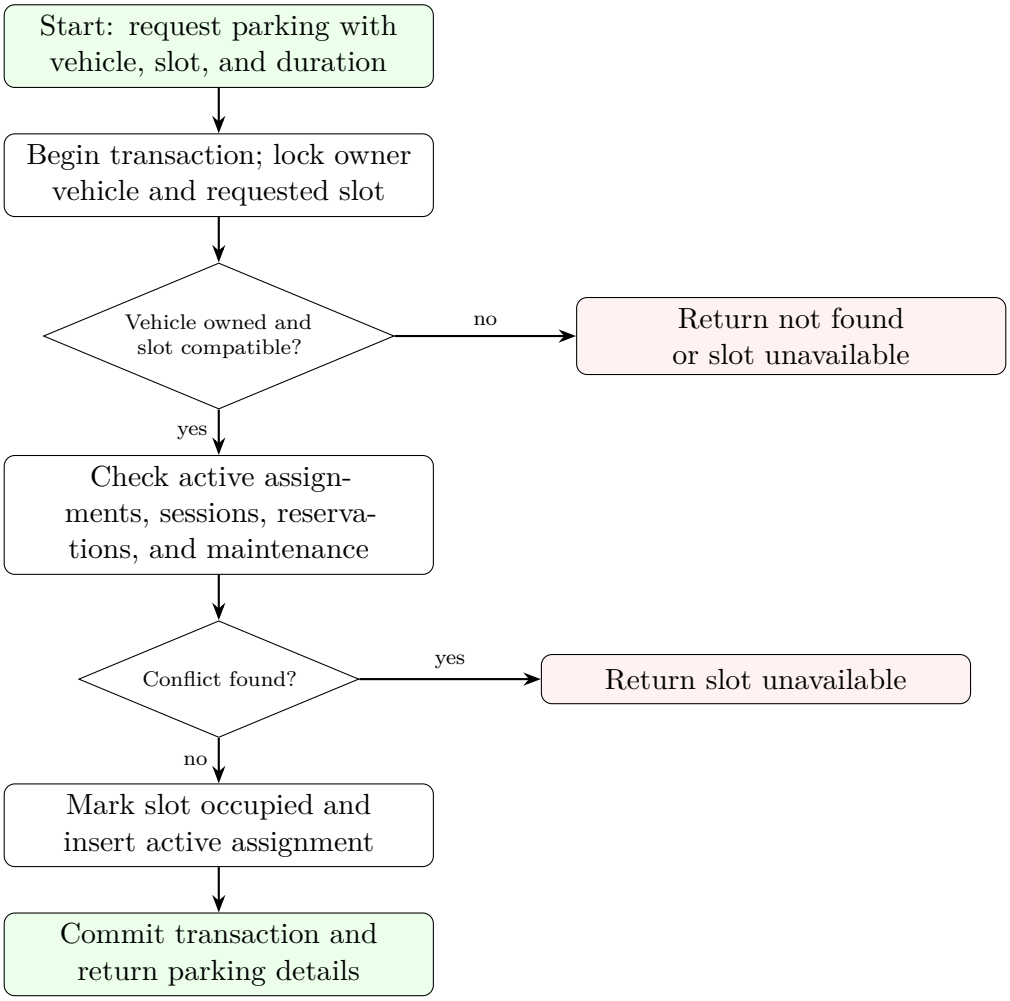


Figure 4: Activity diagram for a direct parking assignment.

17.4.6 Use Case Diagram

Figure 5 summarizes user-facing capabilities. The administrator inherits the ordinary authenticated-user capabilities and adds management operations; the diagram uses generalization for actor roles, not for C++ classes.

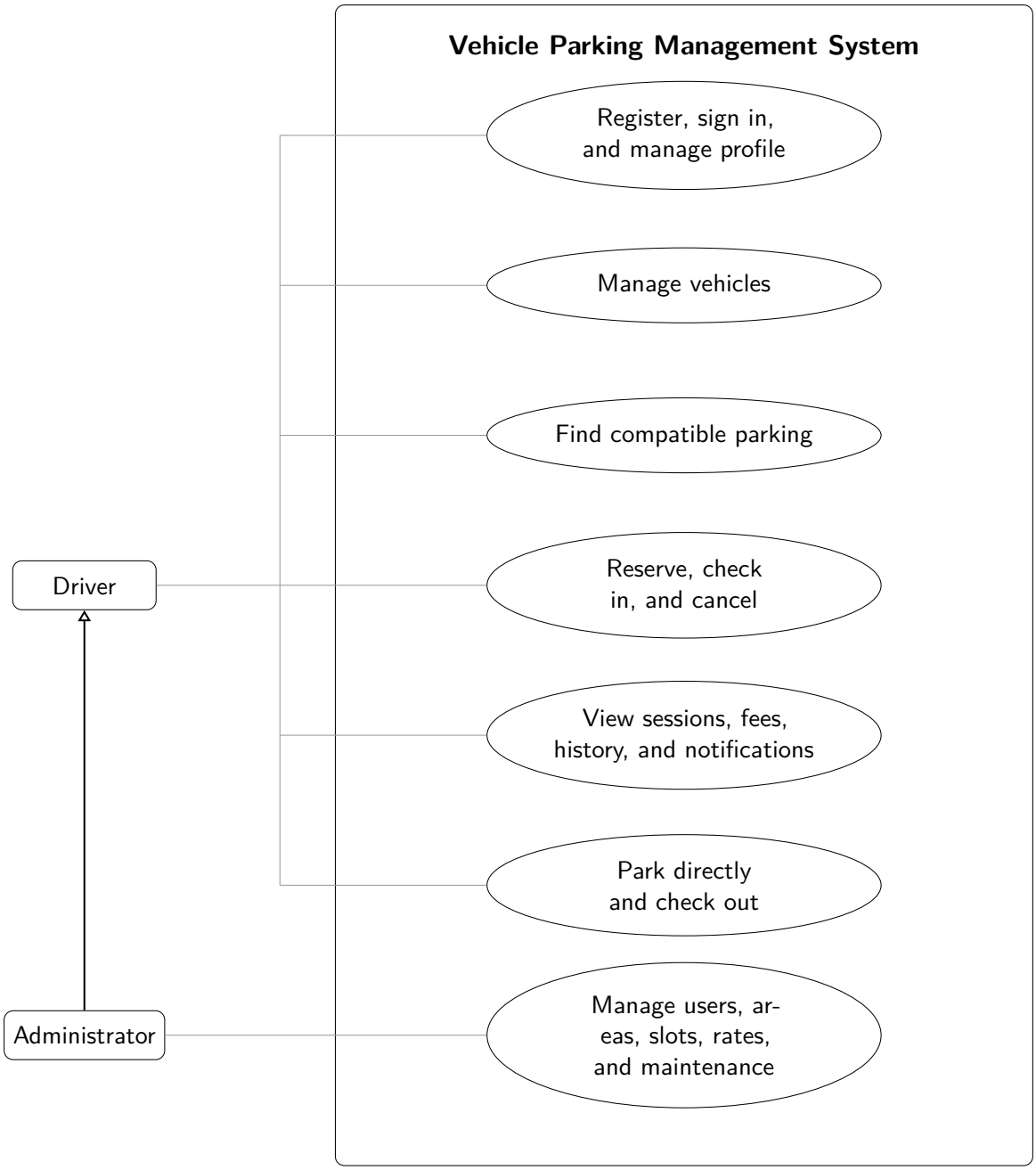


Figure 5: Use case diagram for driver and administrator roles.

17.4.7 OOP Code Examples

Listing 1 shows a project class declaration. The private `database_` reference demonstrates encapsulation and its association with the shared `Database` object.

```
class AuthRoutes
{
public:
    explicit AuthRoutes(Database &database);
    void registerRoutes(crow::App<crow::CORSHandler> &app);

private:
    Database &database_;
```

```
};
```

Listing 1: AuthRoutes class declaration: public operations and private associated database.

Listing 2 is taken from the implemented application constructor. It demonstrates composition: the application owns its database and controller members, and injects the shared database by reference into each controller.

```
ParkingApplication::ParkingApplication(const BackendConfig &config)
    : config_(config),
      database_(config_),
      parking_routes_(database_),
      auth_routes_(database_),
      vehicle_routes_(database_),
      reservation_routes_(database_),
      admin_routes_(database_)
{
}
```

Listing 2: Implemented dependency wiring in ParkingApplication.

The same construction pattern demonstrates constructors and dependency injection: `ParkingApplication` passes its already-created database to the controllers. The application object itself is constructed in `main()` and its public `run()` method hides startup details.

```
void ParkingApplication::run()
{
    configureCors();

    const DatabaseCheckResult database_check = database_.checkConnection();
    if (database_check.connected)
    {
        std::cout << "Database check successful!" << std::endl;
    }
    else
    {
        std::cerr << "Database connection failed: "
                  << database_check.error_message << std::endl;
    }

    registerRoutes(database_check);

    std::cout << "Server running on http://localhost:"
              << config_.server_port << std::endl;
    app_.port(config_.server_port).multithreaded().run();
}
```

Listing 3: Application startup orchestration behind the run() abstraction.

Listing 4 is the project's fee calculation value type and function. It shows a small, cohesive abstraction with an explicit result structure, nonnegative overtime handling, whole-hour booking billing, and currency rounding.

```
struct Breakdown
{
    int billed_hours;
    int billed_overtime_minutes;
    double base_fee;
```

```
    double overtime_fee;
    double total_fee;
};

inline Breakdown calculate(
    int booked_duration_minutes,
    int overtime_duration_seconds,
    double hourly_rate)
{
    const int billed_hours = std::max(1, (booked_duration_minutes + 59) / 60);
    const int nonnegative_overtime_seconds =
        std::max(0, overtime_duration_seconds);
    const int billed_overtime_minutes = nonnegative_overtime_seconds / 60 +
        (nonnegative_overtime_seconds % 60 != 0 ? 1 : 0);
    const double base_fee = roundCurrency(billed_hours * hourly_rate);
    const double overtime_fee = roundCurrency(
        billed_overtime_minutes * overtime_rate_per_minute);

    return {billed_hours, billed_overtime_minutes, base_fee,
            overtime_fee, roundCurrency(base_fee + overtime_fee)};
}
```

Listing 4: Implemented fee calculation in parking_fees.h.

The database workflows use `pqxx::work` transaction objects and parameterized libpqxx calls. On successful direct parking, the implementation updates the slot and inserts an assignment before committing. An early validation return leaves the transaction uncommitted, so transaction scope handles rollback through RAII. No project class uses custom inheritance or virtual methods; this describes the current source accurately. Listing 5 shows RAII and exception handling together: the connection and transaction have local lifetimes, and connection failures are returned as a typed result with an error message.

```
DatabaseCheckResult Database::checkConnection() const
{
    try
    {
        pqxx::connection connection = createConnection();
        pqxx::work transaction(connection);
        transaction.exec("SELECT 1");
        transaction.commit();
        return {true, {}};
    }
    catch (const std::exception &exception)
    {
        return {false, exception.what()};
    }
}
```

Listing 5: Scoped libpqxx resources and exception handling in the database health check.

Startup applies the same exception-handling principle: the top-level handler catches configuration or application failures and exits with an error status. Route handlers similarly translate database exceptions into HTTP error responses.

17.5 Database Design and Persistence

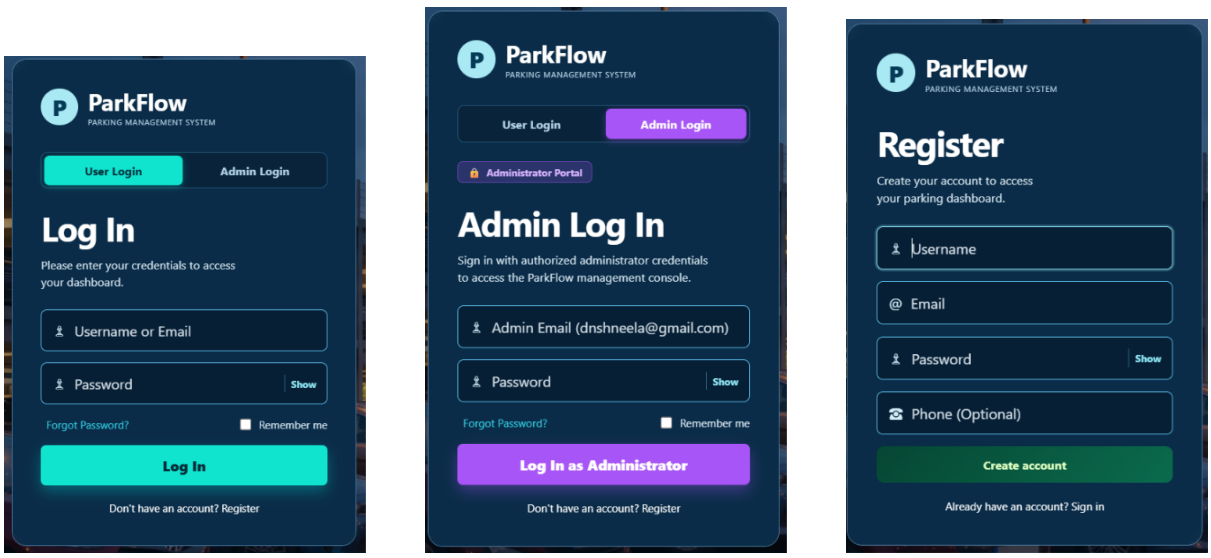
The schema in `database/schema.sql` defines twelve core tables. Identity records are `users`, `auth_sessions`, and `notifications`. Parking resources are `parking_areas`, `parking_slots`, `parking_assignments`, `parking_rates`, and `maintenance`. Reservation and billing records are `reservations`, `parking_sessions`, and `payments`. Primary and foreign keys, check constraints, unique constraints, and indexes enforce data rules. The SQL schema and migration scripts are authoritative for database details.
Reservation and parking operations check ownership and conflicts in database transactions. The schema's partial unique indexes restrict each vehicle and slot to one active direct parking assignment. Reservations and maintenance are considered when searching for eligible slots. User-facing CRUD operations are owner-scoped, while administrator endpoints apply role checks.

17.5.1 Fee Rules

The fee helper bills a minimum of one hour and rounds booked minutes up to whole hours. Overtime duration is rounded up to whole minutes. The implemented constants are a seven-minute free buffer and INR 1 per billed overtime minute; the hourly rate comes from the applicable database rate, with the documented default of INR 40 where a configured reservation rate is absent. A session's fee summary includes booked duration, rate, overtime, and total fee.

17.6 Project Interface and Execution Outputs

The frontend source provides a ParkFlow interface with an authentication view, parking area and slot display, vehicle and reservation workflows, session history, notifications, and an administrator console. Figures 6–16 reproduce the browser screenshots supplied with the project. Each image is stored in the local `report_screenshots/` directory so the report can be compiled without access to the original OneDrive folder.



User login Administrator login Account registration

Figure 6: ParkFlow authentication and registration screens.

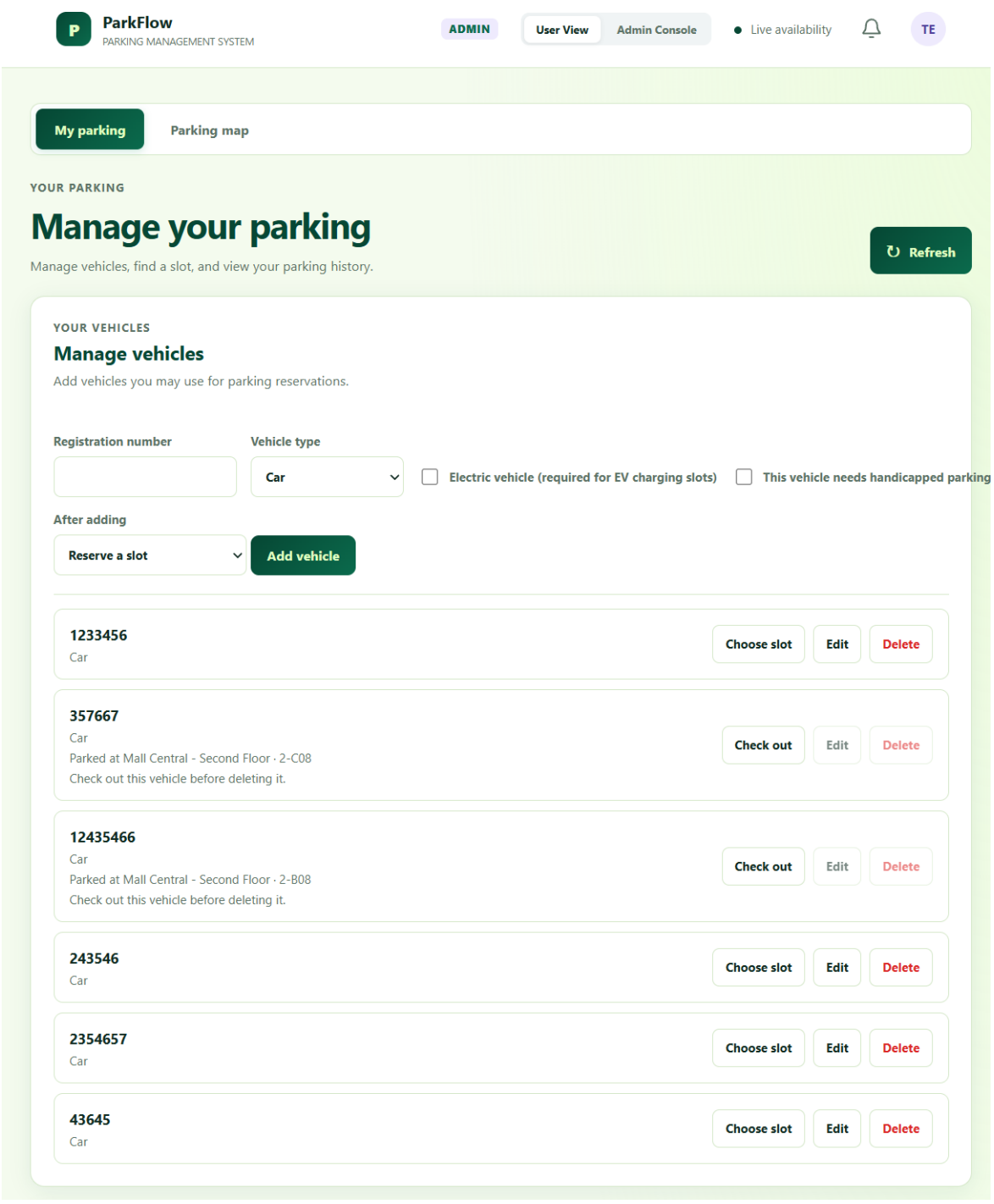


Figure 7: Driver view: Manage Vehicles.

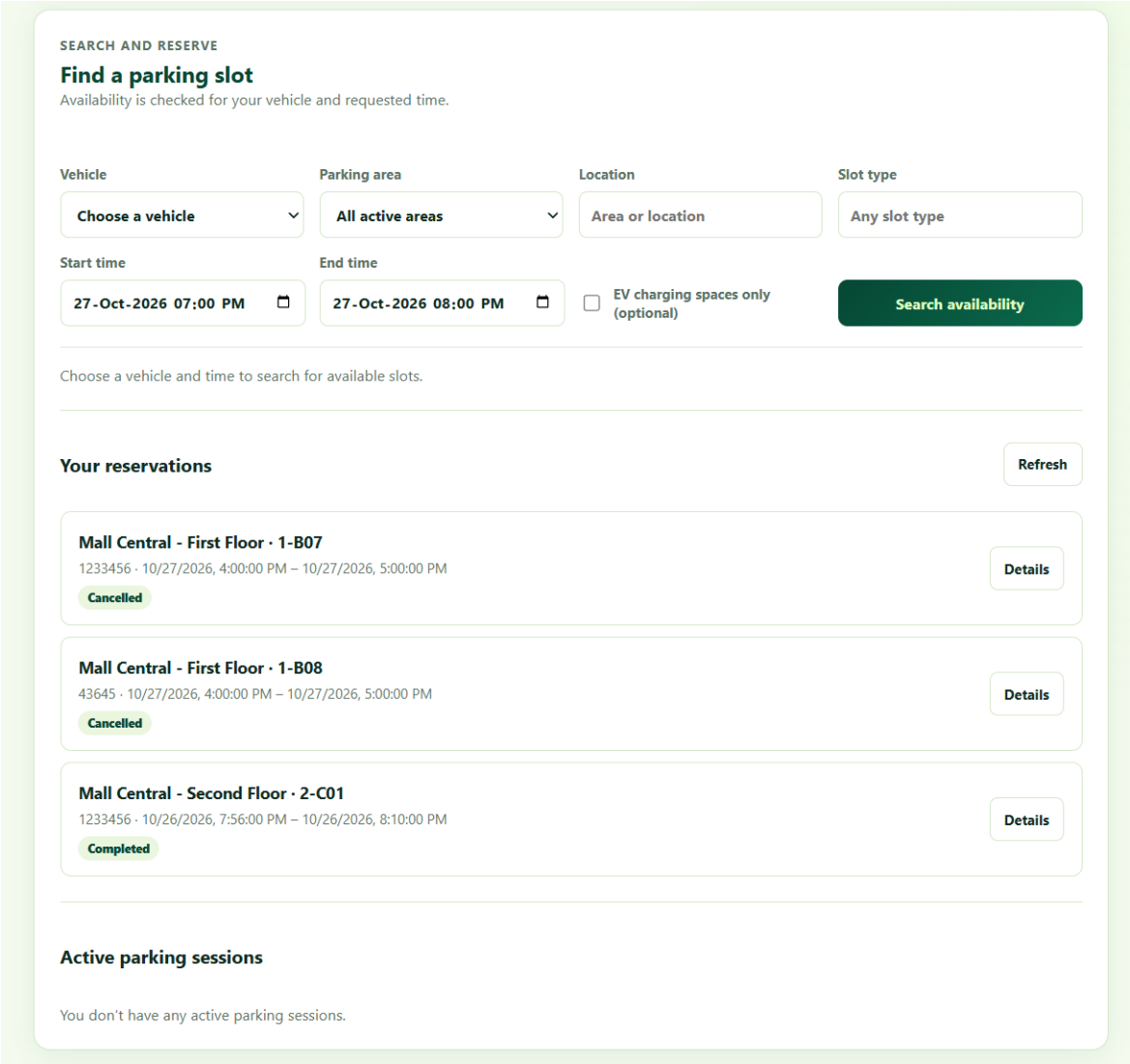


Figure 8: Driver view: Find a Parking Slot, reservations, and active sessions.

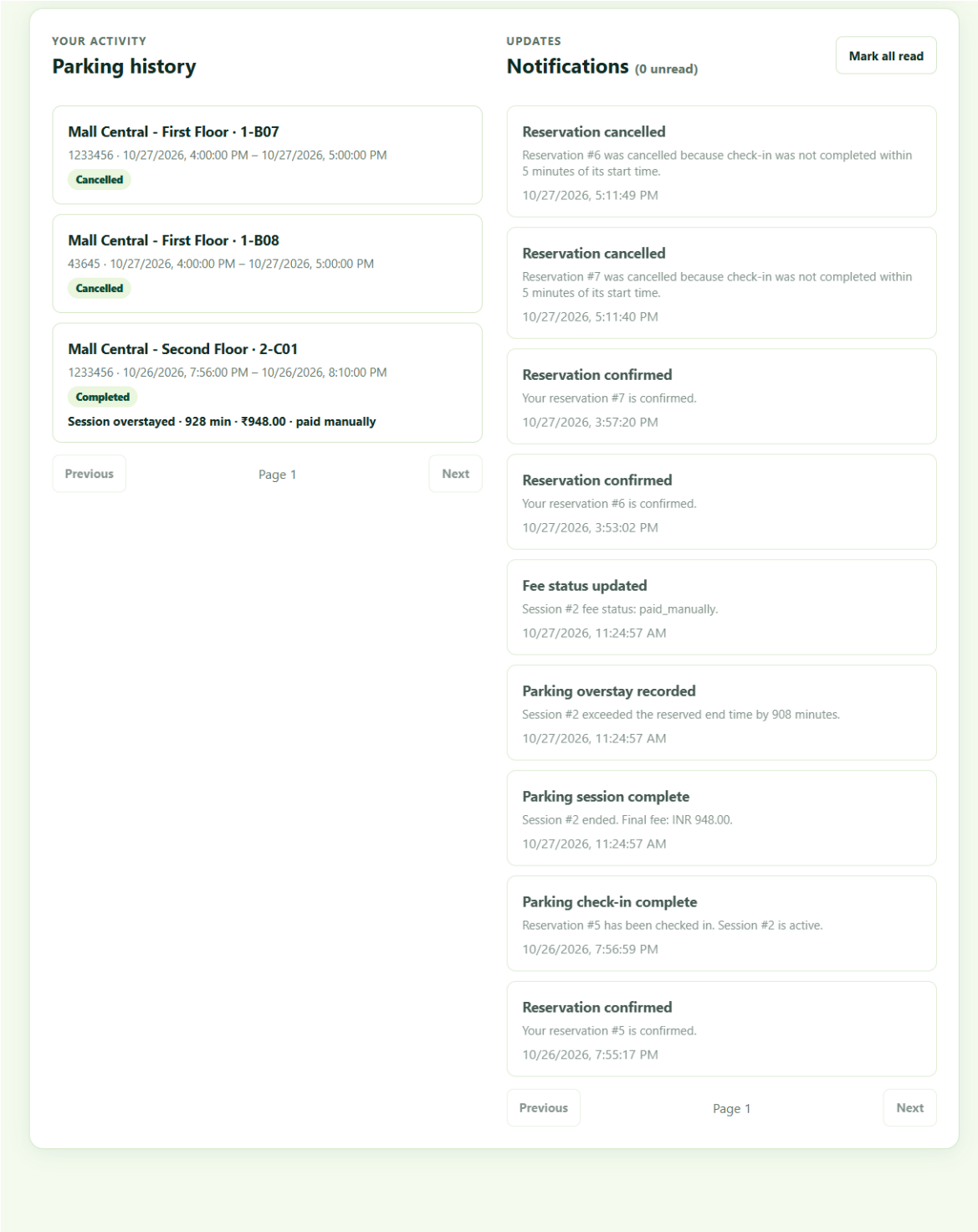


Figure 9: Driver view: Parking History and Notifications.

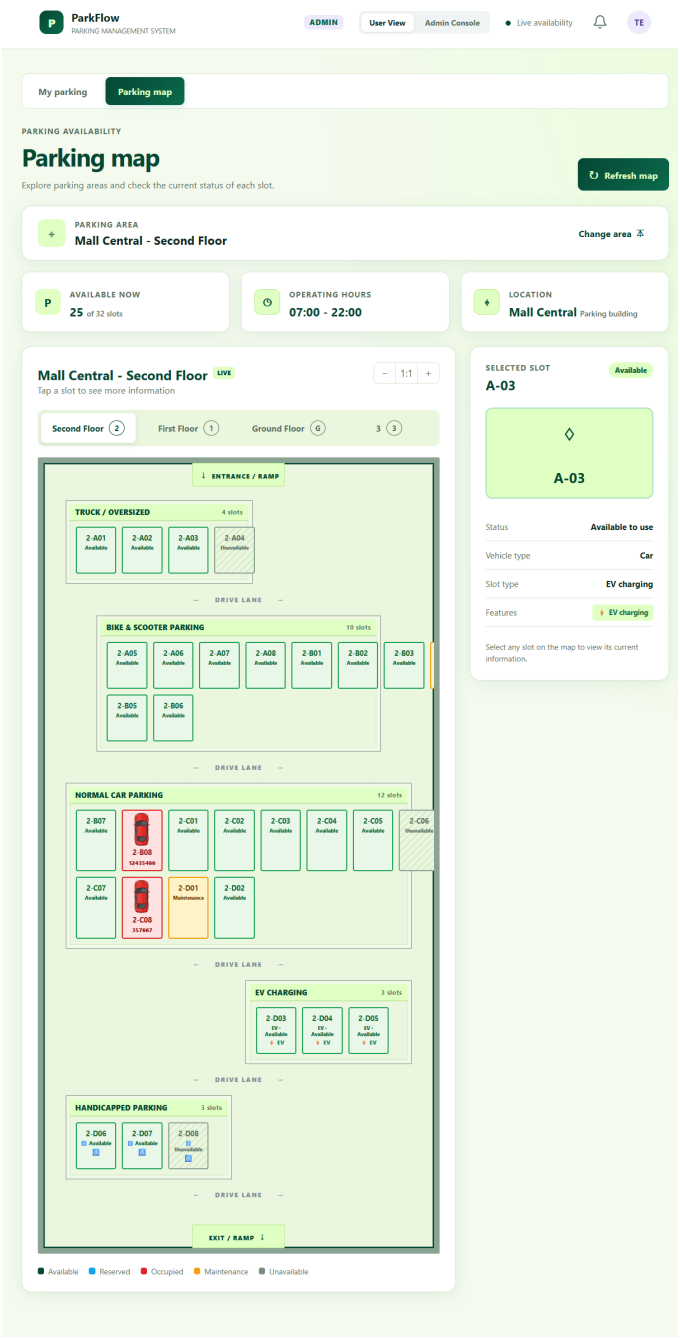


Figure 10: Parking map with area selection, slot availability, and selected-slot details.

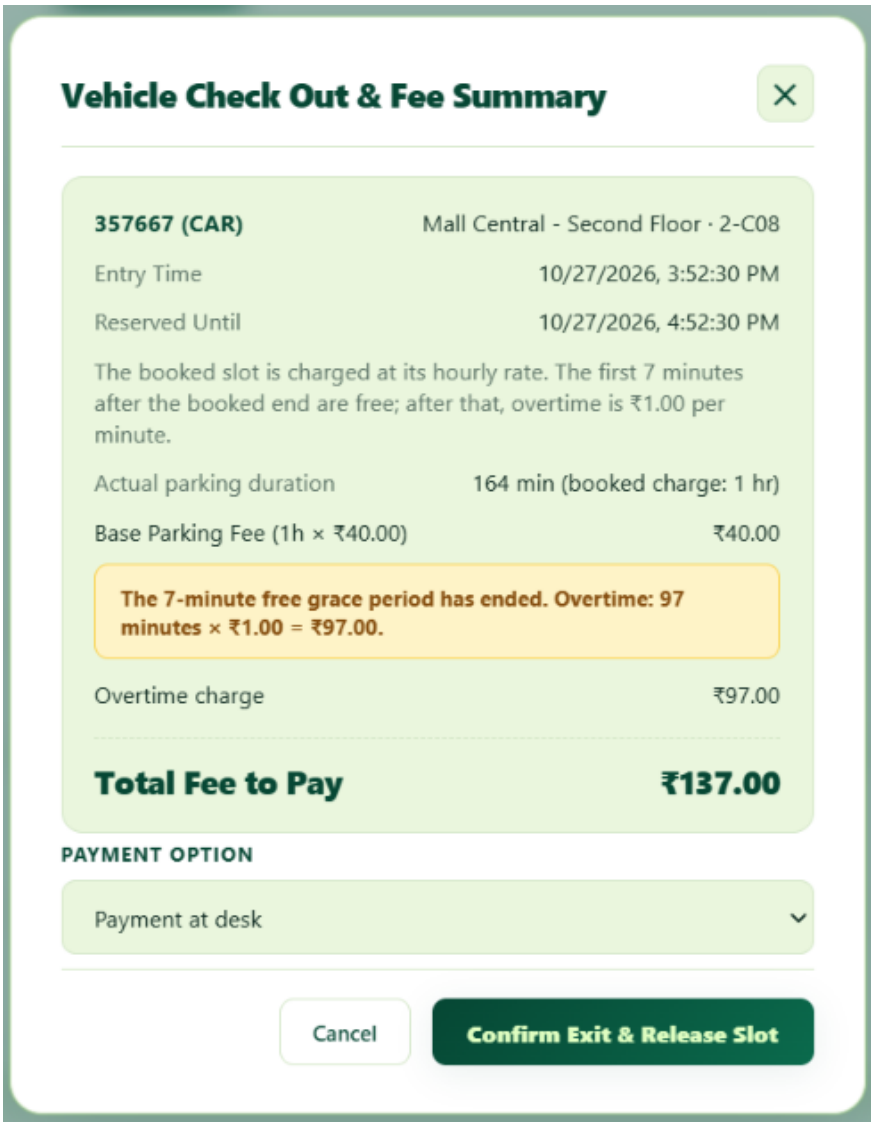


Figure 11: Vehicle checkout fee summary with booked duration, overtime, and payment option.

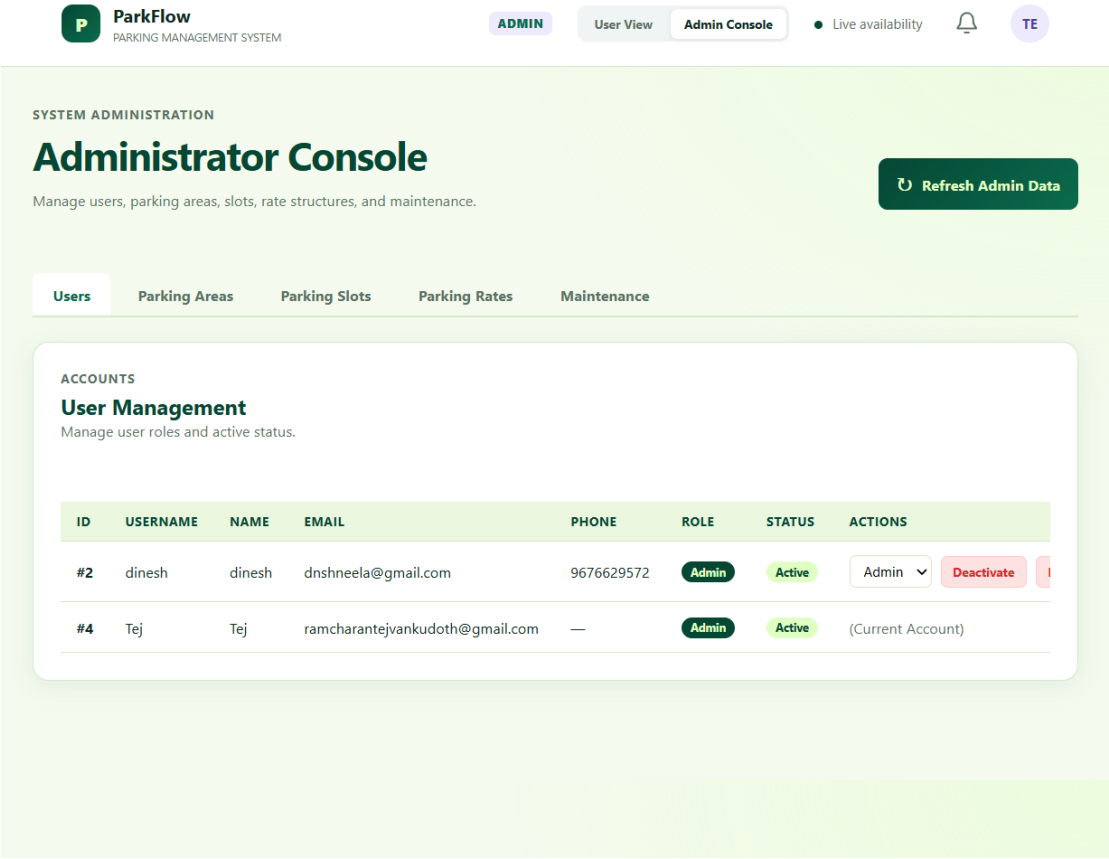


Figure 12: Administrator user-management screen with role, status, and account actions.

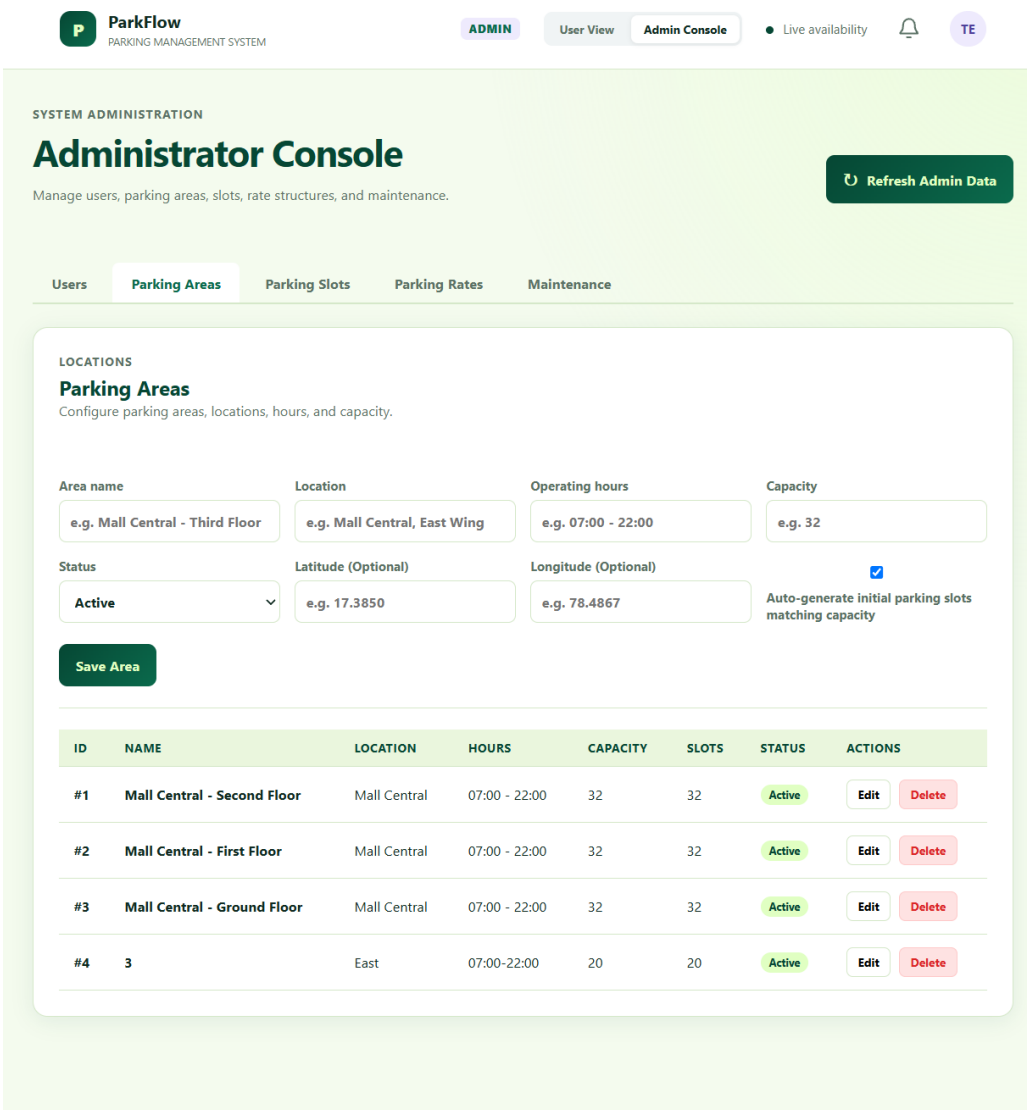


Figure 13: Administrator parking-area configuration and area list.



Figure 14: Administrator slot configuration and slot inventory.

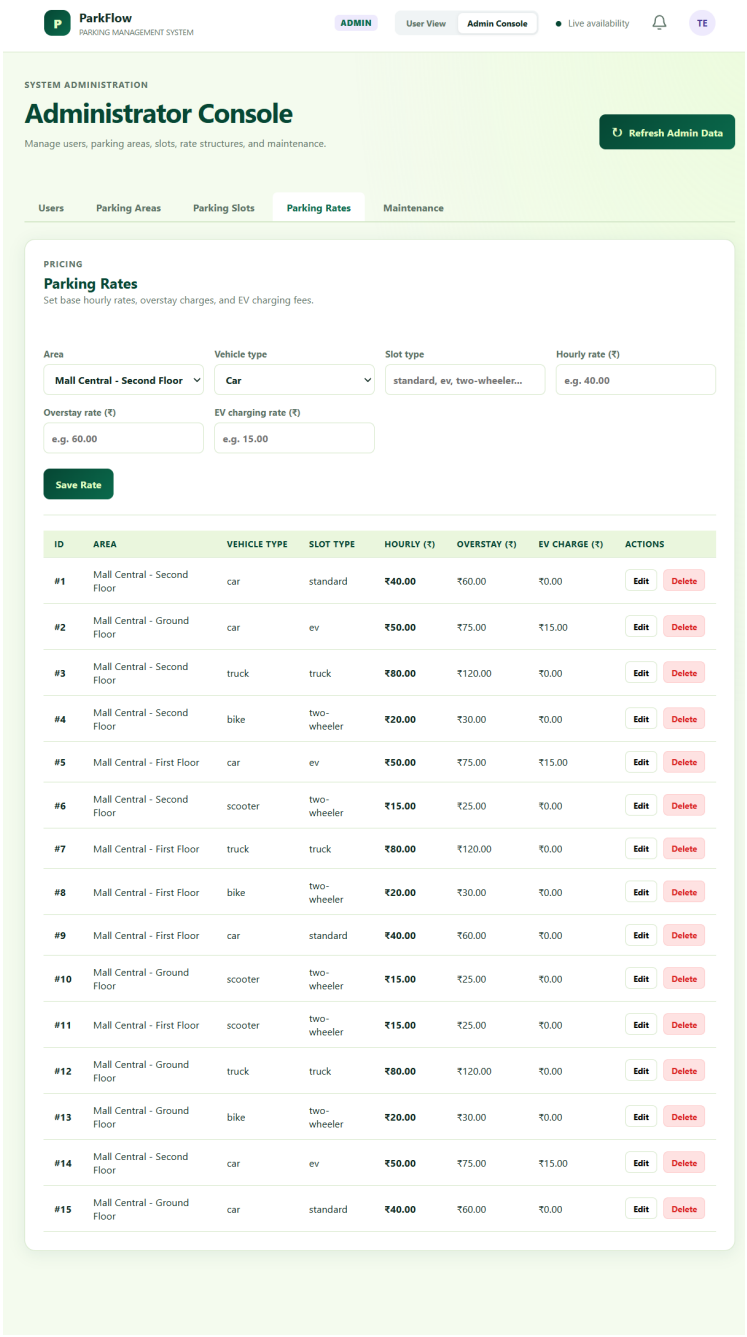


Figure 15: Administrator parking-rate configuration and rate list.

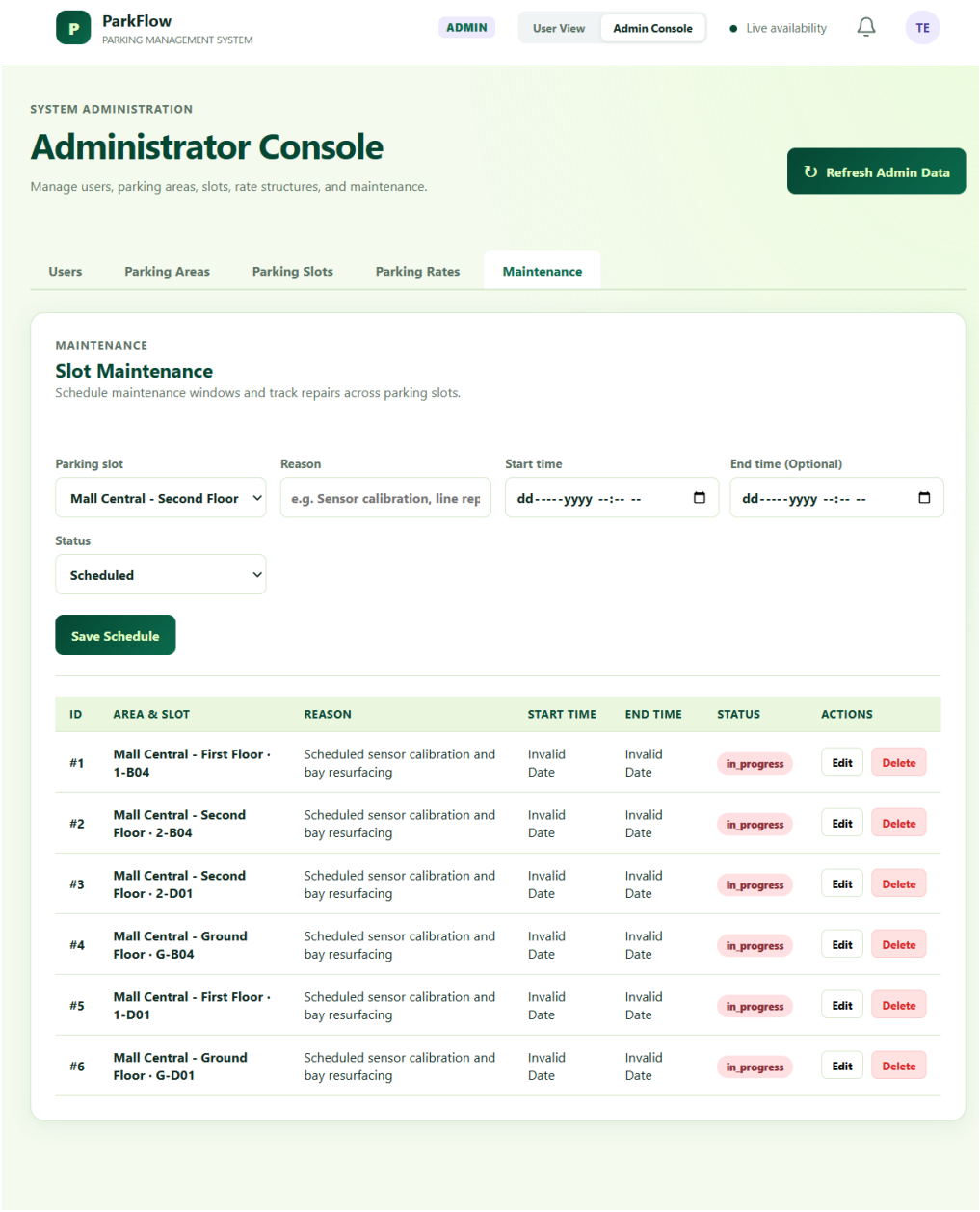


Figure 16: Administrator maintenance scheduling and maintenance list.

17.6.1 Build and Run

The backend build is configured through CMake (minimum version 3.20) and requires a C++20 compiler, Crow, libpqxx, OpenSSL, and PostgreSQL. The frontend can be served as static files; it expects the backend at `http://localhost:3000` by default. The database password is supplied through `PARKING_DB_PASSWORD`; secrets should not be included in the report or committed to source control. Setup and migration commands are documented in `README.md`.

17.6.2 Project Source Map

Path	Responsibility
`backend/src/application/`	Application lifecycle, CORS configuration, dependency wiring.
`backend/src/routes/`	Auth, parking, vehicle, reservation, and administrator HTTP routes.
`backend/src/database/`	PostgreSQL access, transactions, parking workflows, and fee calculations.
`backend/src/config/`	Environment-based backend configuration.
`backend/src/auth/` and `validation/`	Password/token functions and request validation helpers.
`backend/src/api/`	Response value types, JSON serializers, and API errors.
`frontend/`	HTML interface, styles, JavaScript modules, and image assets.
`database/`	Schema, migrations, and development seed scripts.

Table 3: Source tree responsibilities.

17.7 Limitations and Future Work

The implementation status recorded by the project documentation identifies automated reservation expiry, QR-based check-in, online payment processing, refunds, digital receipts, advanced fee rules, external notifications, and advanced reporting as planned work. The repository also records live Phase 8 database end-to-end verification and production deployment as outstanding. These items should not be presented as completed functionality.

17.8 Conclusion

VPMS provides a browser-based parking workflow backed by a C++20 Crow API and PostgreSQL. Its object-oriented structure uses an application composition root, feature-focused route controllers, and an encapsulated database layer. Transactional persistence supports reservations, parking assignments, sessions, billing summaries, history, notifications, and administration. The design demonstrates practical encapsulation, abstraction, composition, and RAII while keeping the report aligned with the current implementation.